BABA IS YOU — Technical Architecture Report

Project LOA 2025–2026 | **Authors:** CHEN Michel, HUYNH Minh Van Claude

1. Scope & Implemented Features

- **Core Game Mechanics:** Implementation of a 2D grid-based movement and interaction system supporting core entities (`BABA`, `WALL`, `ROCK`, `FLAG`) and properties (`YOU`, `STOP`, `PUSH`, `WIN`) linked via the `IS` operator.
- **State History Management:** Full support for an `Undo / Redo` system allowing smooth step-by-step navigation through game actions.
- **Architecture:** Strict separation of concerns adhering to the Model-View-Controller (MVC) pattern, enhanced by custom operator overloading (`board[position]`) for intuitive spatial querying of entities.
- **Extended Features:**
  - `PULL` mechanic (pulling adjacent objects and text blocks).
  - `KEY` and `DOOR` interactions (`SHUT` / `OPEN`).
  - `WALL IS BABA` dynamic transformations.
  - Complex rule evaluation using `AND`.
  - Multi-level progression system.

2. Significant Technical Aspects

- **Grid Data Structure:** The board is modeled as a 2D vector of `Cell` objects (`std::vector<std::vector<Cell>>`), enabling multiple entities to occupy a single grid coordinate simultaneously.
- **Dynamic Rule Engine:** Rule parsing and evaluation occur in real-time based on player actions and active board configurations, allowing dynamic updates to entity physics and behaviors.
- **Design Patterns:**
  - **MVC Pattern:** Isolates core business logic from graphical rendering, coordinated via dedicated controllers.
  - **Singleton Pattern:** Ensures a unique, globally managed instance to minimize conflicts and optimize performance.
  - **Iterator Pattern:** Encapsulates collection traversal without exposing internal storage structures.
  - **Memento Pattern:** Captures and restores entity states efficiently to power the `Undo / Redo` mechanism.
  - **Factory Pattern:** Decouples object instantiation from JSON map parsing, cleanly generating `Entity` instances.

    - **Template Method:** Allows subclasses to specialize behaviors while enforcing a standardized structure across `Entity` types.

3. Known Limitations

- **Graphical Rendering of Overlays:** While game logic correctly handles multiple objects on a single coordinate (e.g., `ROCK` and `BABA`), the current view rendering layer displays only the topmost entity rather than rendering a combined visual representation, diverging slightly from the original game's visual stacking behavior.

4. Unimplemented Extensions

- **Control Remapping (`A IS LEFT`):** Due to time constraints late in the development cycle, the dynamic control remapping extension was omitted.
    - *Proposed Architecture:* This feature would be integrated by having the `KeyboardListener` controller query active rules on the board. If a rule matching `KEY IS DIRECTION` is detected, input interpretation maps to the redefined direction; otherwise, default controls apply.